

Stack Applications: Well-Formedness of Parentheses

Q. Explain how a stack can be used to check the well-formedness of parentheses in an expression. Write a step-by-step algorithmic approach.

> **Definition to Remember**

> An expression is well-formed if every opening bracket has a corresponding closing bracket.

1. Why Stack Works for This Problem

- LIFO ensures the **most recently opened bracket** is at the top of the stack.

- When a closing bracket is encountered, it must match the top of the stack.

2. Algorithm

1. Initialize an empty stack.

3. Trace Example: `{ [ ( ) ] }`

Character	Action	Stack State
{	Push	{
[	Push	{, [
(	Push	{, [, (
)	Pop	{, [,]
]	Pop	{, }
}	Pop	

| :--- | :---

Invalid Example: `{ ( ) }`

- Reason: The closing bracket `)` does not match the top of the stack `{`.

> **Must-Write Points (for 10 marks)**

> 1. Stack is empty initially.

> **Quick Recall**

> `Ope
O(n)`

C Code: Parentheses Checking Algorithm using Stack

```c

#includ

#define MAX 100

struct CharStack {

char ite

void push(struct CharStack \*s, char c) {

s->iter

char pop(struct CharStack \*s) {

if (s->t

int isMatchingPair(char char1, char char2) {

if (char

int checkBalanced(char exp[]) {

struct t

for (int i = 0; i < strlen(exp); i++) {

if (expl

```
int main() {
```

```
char e;
```

```
printf("Expression: %s -> %s\n", expr1, checkBalanced(expr1) ? "Balanced " : "Unbalanced ");
```

```
printf(" ");
```

```
return 0;
```

```
}
```